UrbanFix AI — Full Application System Report

Date: 2026-03-04
Previous revision: 2026-02-20
Workspace: `appv1`
Scope: End-to-end documentation of the currently built application — frontend, backend, data model, configuration, operational flows, AI layer, dev tooling, and every file in the repository.

---

1) Executive Summary

UrbanFix AI is a civic issue reporting and municipal collaboration platform comprising:

- A **React Native (Expo SDK 54)** mobile app (`frontend/`) for citizens, admins, and field workers.
- A **Node.js + Express 5** REST API (`backend/`) backed by **Supabase PostgreSQL** with **PostGIS**.
- **JWT-based application auth** plus **Supabase Auth integration** (Email OTP, Magic Link, Google OAuth).
- **Firebase Cloud Messaging (FCM)** for native push notifications.
- **Role-based UX and permissions** for: `citizen`, `admin`, `super_admin`, `department_admin`, `field_worker`.
- Built-in subsystems for issue lifecycle tracking, map-based discovery, social interactions (upvotes / comments / follows), municipal page publishing, gamification (points / badges / levels / leaderboard), push & in-app notifications, and a promotional engagement scheduler.
- A **mock AI layer** (3 services) with a detailed 12-model AI/ML plan documented for future replacement.

This document describes **how the app is built today**, what every module does, and what is expected for each role and use case.

---

2) Product Goals & Operating Model

2.1 Core Product Goal

Enable citizens to report civic issues quickly and transparently while giving authorities structured workflows to assign, resolve, and communicate updates.

2.2 Main Product Pillars

1. **Report with context** — title, category, media (photo/video), GPS location, anonymous/emergency toggles.
2. **Route and prioritize** — severity score, category, department tagging, admin queue.
3. **Track progress** — status timeline, resolution proof (before/after + worker remarks).
4. **Engage community** — feed interactions, issue/page following, reels/stories UX.
5. **Incentivize participation** — points, 10-level progression, badges, leaderboard.

2.3 Current Role Model

Role	Capabilities

Role	Capabilities
Citizen	Report issues, engage with feed, follow issues/municipal pages, track updates, earn points & badges.
Admin / Super Admin	Triage priority queue, update statuses, assign departments/workers, create & manage municipal pages, publish official posts.
Department Admin	Same as admin scoped to a department (role value exists in schema, UX currently identical to admin).
Field Worker	Receive assigned tasks, mark progress, upload proof, resolve field tasks.
Municipal Page (entity)	Content-author profile represented in feed/posts; pages are created and managed by admins.

3) System Architecture

3.1 High-Level Architecture

```
┌──────────────────────┐
│  Expo React Native   │  Mobile Client
│  (TypeScript)        │  Role-aware navigation
└──────────────────────┘
          │ Axios (JWT Bearer)
┌──────────▼───────────┐
│  Express 5 API       │  backend/server.js
│  Route modules       │  7 route files
│  Auth middleware      │  JWT + role guards
└──────────────────────┘
          │ Supabase JS Client
┌──────────▼───────────┐
│  Supabase PostgreSQL  │  PostGIS enabled
│  + Supabase Auth      │  OTP / OAuth / Magic Link
│  + Storage            │
└──────────────────────┘
          │
┌──────────▼───────────┐
│  Firebase Admin SDK   │  FCM push notifications
└──────────────────────┘
```

3.2 Runtime / Deployment Notes

- Backend default port: **5000** (configurable via `PORT` env var).
- Frontend API base URL selection:
  - Dev: LAN IP `192.168.0.100` (physical device over Wi-Fi).
  - Prod: `https://urban-fix-ai.onrender.com`.
  - Controlled by `__DEV__` flag in `src/services/api.ts`.
- Root `package.json` scripts are deployment-oriented (Render): `start` → `cd backend && npm install && npm start`.

- Engine requirement: Node ≥ 18.0.0.

3.3 Technology Stack

Frontend

Technology	Version	Purpose
React Native	0.81.5	Core framework
Expo SDK	54.0.33	Managed workflow, OTA updates
TypeScript	~5.9.2	Type safety
React Navigation	Stack 7.7 + Tabs 7.14	Navigation
Axios	1.13.5	HTTP client
Supabase JS	2.97.0	Auth (OTP, OAuth)
expo-notifications	0.32.16	Push notification registration
expo-location	19.0.8	GPS acquisition
expo-image-picker	17.0.10	Photo/video capture
expo-camera	17.0.10	Camera access
expo-auth-session	7.0.10	OAuth flow
expo-linear-gradient	15.0.8	Gradient effects
expo-blur	15.0.8	Glass morphism
react-native-maps	1.20.1	Map view
react-native-reanimated	4.1.1	Animations
react-native-gesture-handler	2.28.0	Gesture interactions
lucide-react-native	0.574.0	Icon set
nativewind / tailwindcss	4.2.1 / 3.4.19	Styling
@expo-google-fonts/inter	0.4.2	Typography

Backend

Technology	Version	Purpose
Node.js	≥ 18	Runtime
Express	5.2.1	HTTP framework
Supabase JS	2.49.4	DB client (service role)
jsonwebtoken	9.0.3	JWT generation/validation

Technology	Version	Purpose
bcryptjs	2.4.3	Password hashing
firebase-admin	13.6.1	FCM push delivery
multer	2.0.2	File upload handling
pg	8.18.0	Direct PostgreSQL client (migrations)
dotenv	17.3.1	Environment variables
cors	2.8.6	Cross-origin support
nodemon	3.1.11	Dev hot-reload

Database

- Supabase PostgreSQL with **PostGIS** extension for geospatial queries (`GEOGRAPHY(Point, 4326)`, `GIST` index).
- Supabase Auth for OTP / Magic Link / Google OAuth.

---

4) Repository Structure and Responsibilities

4.1 Root Level

File	Purpose
`package.json`	Monorepo root — deployment scripts (`start`, `build`), Node engine constraint.
`app.json`	Root Expo config — EAS project ID, Android package `com.ezzshashiiii.urbanfixroot`.
`eas.json`	EAS Build profiles: `development` (dev client, internal), `preview` (internal), `production` (auto-increment). CLI ≥ 18.0.1, app version source `remote`.
`.gitignore`	Git ignore rules.

4.2 Backend (`/backend`)

Core

File	Purpose
`server.js`	Express bootstrap — middleware stack, route mounting, Firebase init, promo scheduler start.
`package.json`	Dependencies, scripts (`start`, `dev`, `seed`, `build`).
`.env`	Environment variables (not committed).
`eas.json`	Empty placeholder.

Routes (`routes/`)

File	Mount Point	Description
`authRoutes.js`	`/api/auth`	Register, login, Supabase login.
`issueRoutes.js`	`/api/issues`	CRUD issues, feed, municipal feed, upvote/downvote, follow, comments, seed-nearby, GeoJSON, seen tracking.
`workflowRoutes.js`	`/api/workflows`	Status update, assignment, worker update, assigned tasks.
`gamificationRoutes.js`	`/api/gamification`	Leaderboard, badges, platform stats.
`notificationRoutes.js`	`/api/notifications`	CRUD notifications, read state, unread count, delete all.
`municipal.js`	`/api/municipal`	Page CRUD, search, suggested, follow/unfollow, post creation, page update.
`userRoutes.js`	`/api/users`	Username check, profile get/update, push-token registration.

Middleware (`middleware/`)

File	Purpose
`authMiddleware.js`	JWT validation (`protect`), role guards (`admin`, `fieldWorker`). Normalizes user ID/role onto `req.user`.
`requestLogger.js`	Colorized request/response logger — method, path, body (passwords masked), query params, authenticated user, response status, duration (ms).

Data Layer (`data/`)

File	Purpose
`store.js`	**Complete Supabase data access layer** (772 lines, 48 exports). All DB operations are async. Includes retry wrapper (`withRetry`, 2 retries, exponential backoff). JWT generation (30-day expiry). 10-level gamification reference data.
`seed.js`	Base database seeder — populates initial users, issues, badges, etc.

Config (`config/`)

File	Purpose
`supabase.js`	Creates Supabase client using service role key. Validates env vars on startup.

File	Purpose
`firebase.js`	Firebase Admin initialization — supports env-var credential (production) or local `serviceAccountKey.json` (dev). Exports `initFirebase()` and `getMessaging()`.
`multer.js`	Multer disk storage config — uploads to `public/uploads/`, 50 MB limit, image/video filter only, random filenames.
`db.js`	Legacy Mongoose/MongoDB connector (currently unused — app uses Supabase).
`serviceAccountKey.json`	Firebase service account credentials (local dev).

Services (`services/`)

File	Purpose
`notificationService.js`	`NotificationService` class (226 lines) — stores in-app notifications in DB via `store.createNotification()`, retrieves user push tokens, filters out Expo tokens (keeps native FCM only), delivers via `sendEachForMulticast` (Firebase Admin v13+). Includes `sendPromoNotification()` for engagement messages.
`promoScheduler.js`	Promotional engagement notification scheduler (171 lines). 12 engagement message templates. Sends to a random 20% of active users, 2–3 times/day during hours 9–20. Self-reschedules at midnight. Exports `startPromoScheduler()`, `stopPromoScheduler()`, `sendDailyPromoToRandomUsers()`.

AI Services (`services/ai/`)

File	Purpose	Status
`detectIssue.js`	Issue detection in images — returns random category (Pothole/Garbage/StreetLight/Graffiti) with 95% confidence.	**MOCK**
`severityScore.js`	Severity scoring — static category→severity map (Pothole→Medium, Garbage→Low, StreetLight→High, Graffiti→Low).	**MOCK**
`duplicateCheck.js`	Duplicate detection — always returns `false`.	**MOCK**

Models (`models/`) — Legacy Mongoose Schemas

These are Mongoose model definitions from the pre-Supabase era. They still exist in the repository but **are not used at runtime** — all data operations go through `data/store.js` → Supabase.

File	Schema	Notable Fields

File	Schema	Notable Fields
`User.js`	`User`	name, email, password, role (citizen/admin/department_admin/field_worker/super_admin), points, badges, reportsResolved, impactScore, region, avatar.
`Issue.js`	`Issue`	user, municipalPage, authorType (User/MunicipalPage), officialUpdateType, title, description, image, mediaProof, location (GeoJSON Point), departmentTag, status, statusTimeline, priorityScore, aiSeverity, aiTags, upvotes, resolvedBy, resolutionProof. 2dsphere index.
`Comment.js`	`Comment`	user, issue, text, parentComment (nested comments).
`Follow.js`	`Follow`	followerId → User, followingId → MunicipalPage, notificationsEnabled. Compound unique index.
`MunicipalPage.js`	`MunicipalPage`	name, handle, department, region (city/ward), verified, followersCount, avatar, coverImage, description, createdByAdminId, contactEmail, isActive, pageType (Department/City/EmergencyAuthority).
`Gamification.js`	`Badge` + `Reward`	Badge: name, description, icon, criteria. Reward: user, points, reason.

SQL Migrations

File	Purpose
`supabase_migration.sql`	**Core schema** (244 lines) — creates all primary tables (users, levels, badges, municipal_pages, issues, issue_upvotes, issue_downvotes, issue_followers, status_timeline, resolution_proofs, comments, notifications, follows, rewards), PostGIS extension, indexes, `update_updated_at` triggers.
`migration_push_tokens.sql`	Creates `push_tokens` table with user FK, token, device_type (android/ios/web), unique token constraint, auto-update trigger.
`migration_performance_indexes.sql`	Adds performance indexes for feed/workflow queries on issues, upvotes, downvotes, followers, follows, comments, status_timeline.
`migration_municipal_feed_seen.sql`	Creates `municipal_post_seen` table for seen/unseen tracking in municipal feed.
`add_username_columns.sql`	Adds `username`, `city`, `ward`, `interests` columns to users table with unique/lower-case indexes.

Utility / Seeder Scripts

File	Purpose
`run_migration.js`	Runs `supabase_migration.sql` programmatically against Supabase (128 lines).
`run_push_migration.js`	Runs `migration_push_tokens.sql` against Supabase (74 lines).
`seed_community_posts.js`	Seeds community issue posts, copies image assets from `frontend/assets/` to `public/uploads/` (119 lines).
`seed_municipal.js`	Seeds municipal pages (Boisar Municipal Council, Palghar Zilla Parishad, Roads Dept, etc.) with department, region, description, contact info (308 lines).
`check_tokens.js`	Queries and prints all records in `push_tokens` table for debugging.
`debug_notifications.js`	End-to-end push notification diagnostic — checks Firebase init, queries push tokens, validates pipeline.
`test_push.js`	Sends a test push notification to a hardcoded user ID via `NotificationService.sendToUser()`.

Static Assets (`public/`)

- `public/uploads/` — uploaded media files (images/videos for issue reports).
- `public/images/` — static image assets (e.g., `garbage.avif`).

4.3 Frontend (`/frontend`)

Entry Files

File	Purpose
`App.tsx`	Root component — font loading (Inter family: 400/500/600/700/900), splash screen (2.5s), ErrorBoundary + AuthProvider + SafeAreaProvider wrapping, push notification listener registration, global non-fatal error handler, LogBox warning suppressions.
`index.js`	Expo entry — `registerRootComponent(App)`.
`app.json`	Expo config — app name "UrbanFix AI", slug `expo1`, version `1.1.0`, scheme `urbanfix`, iOS bundle `com.urbanfix.ai`, Android package `com.urbanfix.ai`, portrait-only, new architecture enabled, edge-to-edge Android, Google Maps API key, plugins (expo-notifications with background remote, expo-web-browser), deep link intent filter `urbanfix://auth/callback`, EAS project ID.
`eas.json`	EAS Build profiles: `development` (dev client), `preview` (APK, internal), `production` (APK). App version source `local`.
`tsconfig.json`	Extends `expo/tsconfig.base`, custom `typeRoots` (./src/types + @types), `skipLibCheck: true`.
`google-services.json`	Firebase/Google services config for Android.

File	Purpose
`package.json`	Frontend dependencies (see Technology Stack above).

Context (`src/context/`)

File	Purpose
`AuthContext.tsx`	Central auth state manager. Provides: `user`, `loading`, `needsLocationSetup`, `needsProfileSetup`, `userLocation`. Functions: `login`, `register`, `logout`, `loginWithOTP`, `verifyOTP`, `loginWithGoogle`, `updateUserLocation`, `completeLocationSetup`, `completeProfileSetup`, `completeOnboarding`, `refreshProfile`. Persists token/user in AsyncStorage. Integrates Supabase Auth for OTP/OAuth flows.

Navigation (`src/navigation/`)

File	Purpose
`RootNavigator.tsx`	Root stack — 14 screen registrations across 3 conditional branches (unauthenticated → setup → main app). Role-based main tabs selection.
`CitizenTabs.tsx`	Citizen tab bar — Feed, Map, Report (FAB), Alerts, Profile.
`AdminTabs.tsx`	Admin tab bar — Dashboard, Map, Alerts, Profile.
`WorkerTabs.tsx`	Worker tab bar — Tasks, Map, Alerts, Profile.

Screens (`src/screens/`)

Auth Screens (`Auth/`)

Screen	Purpose
`OnboardingScreen.tsx`	First-launch onboarding carousel.
`LoginScreen.tsx`	Email/password login + social login options.
`RegisterScreen.tsx`	New account registration form.
`MagicLinkScreen.tsx`	Email magic-link sign-in flow.
`OTPVerifyScreen.tsx`	OTP code verification screen.
`LocationSetupScreen.tsx`	GPS permission request + location confirmation.
`ProfileSetupScreen.tsx`	Username, city, ward, interests setup.
`SplashScreen.tsx`	Animated splash / loading screen.

Main Screens (`Main/`)

Screen	Purpose
`HomeFeed.tsx`	Dual-mode feed (Community / Municipal) with filter drawer, reels tab, stories row, feed posts.
`MapScreen.tsx`	Map view with live location, category/status filters, severity markers, heatmap circles, 50km proximity filter.
`ReportIssueScreen.tsx`	Issue report form — category, title, description, photo/video, location (EXIF → GPS → cache), anonymous toggle, emergency toggle.
`IssueDetailScreen.tsx`	Issue detail — media, severity, tags, priority score, status timeline, resolution proof, social actions, comments, follow issue.
`NotificationsScreen.tsx`	Notification center — polling, swipe-to-delete, mark read, badge count.
`ProfileScreen.tsx`	User profile — stats, badge browser, level progress, logout.
`LeaderboardScreen.tsx`	Citizen ranking by points.
`SettingsScreen.tsx`	Settings toggles (dark mode), support/legal links, danger zone actions.
`EditProfileScreen.tsx`	Profile editing form.

Admin Screens (`Admin/`)

Screen	Purpose
`AdminDashboard.tsx`	High-priority queue, status updates (Acknowledged/InProgress/Resolved), department/worker assignment, aggregate platform stats.

Worker Screens (`Worker/`)

Screen	Purpose
`FieldWorkerDashboard.tsx`	Assigned tasks list, start work, complete with proof photo, open map directions.

Municipal Screens (`Municipal/`)

Screen	Purpose
`MunicipalProfileScreen.tsx`	Municipal page profile — Updates/About tabs, follow/unfollow.

Components (`src/components/`)

Feed Components (`feed/`)

Component	Purpose

Component	Purpose
`FeedPost.tsx`	Premium social-media-style post card (401 lines) — double-tap-to-like with heart animation, reanimated + gesture handler, municipal vs community post distinction, memo-wrapped.
`FeedToggle.tsx`	Animated Community ↔ Municipal toggle with spring animation + micro-bounce. Blue gradient (Community) / purple gradient (Municipal).
`FilterDrawer.tsx`	Animated 75% side-drawer with blur overlay for feed filtering and sorting. Spring open / timing close.
`ReelsTab.tsx`	TikTok/Instagram Reels-style vertical swipeable full-screen cards (484 lines). Heart bounce animation, gradient overlays, municipal vs community differentiation.
`StoriesRow.tsx`	Instagram-style horizontal stories row for municipal accounts — circular avatars, gradient rings (blue→purple→pink), staggered entrance animation, "Add Story" button.

Municipal Components (`Municipal/`)

Component	Purpose
`SuggestedFollows.tsx`	Horizontal scrollable "Suggested for You" municipal page row. Fetches from `/municipal/suggested`. Dismissible. Navigates to MunicipalProfile on press.

UI Components (`ui/`)

Component	Purpose
`ErrorBoundary.tsx`	React class-based error boundary (197 lines). Catches JS errors, shows styled error screen with retry. Mentions Sentry/Bugsnag/Crashlytics integration for production.
`FloatingReportFAB.tsx`	Circular 64×64 floating action button using lucide Plus icon, primary color, shadow glow, z-index 1000.
`GlassCard.tsx`	Glassmorphism card wrapper using `expo-blur` BlurView. Configurable intensity, dark tint, frosted-glass border.
`IssueCard.tsx`	Social-media-style issue card (201 lines) — fade-in + slide-up entry animation, severity color coding (Critical=red, High=red-orange, Medium=yellow, Low=green), status icons, linear gradient glow.
`NeonBadge.tsx`	Severity badge with neon glow border. Color-mapped: Critical=#FF0000, High=error, Medium=accent, Low=success.
`PriorityMeter.tsx`	Animated horizontal priority bar (0–10). Color thresholds: green ≤ 4, yellow 5–7, red > 7. 1-second fill animation.

Services (`src/services/`)

File	Purpose
`api.ts`	Central Axios client (237 lines). Auto Bearer token from AsyncStorage. Request/response logging. Cache-busting on GET (`_ts` param + no-cache headers). 20s timeout. Platform-aware base URL. Exports typed API wrappers: `authAPI`, `issuesAPI`, `workflowAPI`, `notificationsAPI`, `userAPI`, `gamificationAPI`, plus `User` interface.
`locationService.ts`	Production-grade GPS service (222 lines). 3 retries with decreasing accuracy (High → Balanced → Low). 10s timeout per attempt. Reverse geocoding. Persistent caching in AsyncStorage. Settings prompt on permission denial.
`notificationService.ts`	Expo push token registration (69 lines). Gets native FCM token via `getDevicePushTokenAsync()`. Validates non-Expo format. Android channel: max importance, vibration pattern `[0,250,250,250]`. Registers with backend.
`supabaseClient.tsx`	Supabase client initialization. AsyncStorage auth persistence. Auto-refresh tokens. `detectSessionInUrl: false`.

Theme (`src/theme/`)

File	Purpose
`colors.ts`	Central design token file. Dark-mode palette: background `#0A0A0F`, primary `#007AFF`, accent `#FFD60A`, error `#FF453A`, success `#30D158`, glass `rgba(20,20,32,0.85)`. Font families: Inter (Regular/Medium/SemiBold/Bold/Black). Spacing scale: xs(4) → xxxl(32). Border radii: sm(8) → full(999).

Types (`src/types/`)

File	Purpose
`expo-vector-icons.d.ts`	TypeScript declarations for `@expo/vector-icons` — exports Ionicons, MaterialIcons, FontAwesome, Feather with `IconProps` interface.

Utils (`src/utils/`)

File	Purpose
`logger.ts`	Dev-only console logger with timestamps, emoji-coded levels (info/warn/error/success/action), category tagging. Convenience: `tap(screen, button)`, `apiReq(method, url)`, `apiRes(method, url, status)`. Only active when `__DEV__` is true.

Android Build (`android/`)

- Native Android project generated by Expo prebuild.
- Contains Gradle wrapper, autolinking, build intermediates, debug + main source sets.

- `google-services.json` at root and Android level for Firebase integration.

Assets (`assets/`)

- `mcd-garbage_d06a446e-cb25-11e8-a159-d4219452a912.avif` — sample issue image used for seeding.

4.4 Docs (`/docs`)

File	Purpose
`FULL_APP_SYSTEM_REPORT.md`	This system report (previous revision 2026-02-20).
`FULL_APP_SYSTEM_REPORT.pdf`	PDF export of the system report.
`AIML_MODELS_PLAN.md`	Comprehensive AI/ML development plan (v2.0, 2026-02-19, 1321 lines). Covers 12 planned models, architecture, training, evaluation, MLOps, costs, and roadmap.

---

5) Authentication, Session, and Onboarding

5.1 Auth Methods Supported

1. **Email + Password** — backend `/api/auth/register` and `/api/auth/login`.
2. **Email OTP / Magic Link** — Supabase Auth via `loginWithOTP()` + `verifyOTP()` in `AuthContext`.
3. **Google OAuth** — Supabase Auth with `expo-auth-session` and deep-link callback (`urbanfix://auth/callback`).

5.2 Session Model

- Supabase-authenticated users call `/api/auth/supabase-login` to exchange for an **app JWT** (30-day expiry, generated in `store.js`).
- App JWT stored in AsyncStorage (`token`), attached to every API request by Axios request interceptor.
- User object persisted as `user` in AsyncStorage for offline session restore.
- Supabase client maintains its own session (auto-refresh enabled) for OTP/OAuth flows.

5.3 First-Run / Setup Gates

After authentication, user flow is gated by `AuthContext` flags:

1. **`needsLocationSetup`** → `LocationSetupScreen` — GPS permission + location confirmation.
2. **`needsProfileSetup`** → `ProfileSetupScreen` — username, city, ward, interests.
3. Both complete → role-based main tabs.

5.4 Profile Setup Expectations

- **Username:** minimum length + character restrictions + uniqueness check via `GET /api/users/check-username/:username`.
- **Location:** city + ward confirmed.
- **Interests:** at least one civic topic selected.

5.5 Onboarding Flow

`OnboardingScreen` → `LoginScreen` (or `RegisterScreen`) → `MagicLinkScreen` / `OTPVerifyScreen` (optional) → `LocationSetupScreen` → `ProfileSetupScreen` → Main App.

---

6) Navigation and Role-Based UX

6.1 Root Navigation Decisioning (RootNavigator.tsx — 14 screens)

```
No user
  ├── Onboarding
  ├── Login
  ├── Register
  ├── MagicLink
  └── OTPVerify

Authenticated + needsLocationSetup
  └── LocationSetup

Authenticated + needsProfileSetup
  └── ProfileSetup

Authenticated + setup complete
  ├── MainTabs (role-based: CitizenTabs / AdminTabs / WorkerTabs)
  ├── IssueDetail
  ├── ReportIssue
  ├── Leaderboard
  ├── Settings
  ├── EditProfile
  └── MunicipalProfile
```

6.2 Citizen Tabs

Tab	Screen	Icon
Feed	HomeFeed	Home
Map	MapScreen	Map
Report	FloatingReportFAB → ReportIssueScreen	Plus (central FAB)
Alerts	NotificationsScreen (badge polling)	Bell
Profile	ProfileScreen	User

6.3 Admin Tabs

Tab	Screen
Dashboard	AdminDashboard

Tab	Screen
Map	MapScreen
Alerts	NotificationsScreen
Profile	ProfileScreen

6.4 Worker Tabs

Tab	Screen
Tasks	FieldWorkerDashboard
Map	MapScreen
Alerts	NotificationsScreen
Profile	ProfileScreen

---

7) Feature Catalogue (Full)

7.1 Issue Reporting

Screen: `ReportIssueScreen`
Capabilities:

- Category selection.
- Title + description.
- Photo/video/gallery evidence capture.
- Location detection priority:
    1. EXIF GPS metadata from selected media.
    2. Live GPS (3 attempts with decreasing accuracy).
    3. Cached location from AsyncStorage.
- Anonymous toggle.
- Emergency toggle.

Backend behavior (`POST /api/issues`):

- Creates issue with mock AI severity classification.
- Initializes status timeline as `Submitted`.
- Rewards points (+10) for non-anonymous reports.
- May trigger first-report badge notification.

7.2 Feed System

Screen: `HomeFeed`
Modes (via FeedToggle):

- **Community feed** — citizen issue posts (blue gradient indicator).
- **Municipal feed** — official page posts with seen/unseen tracking (purple gradient).

Sub-experiences:

- **Reels tab** — TikTok-style vertical swipeable full-screen issue cards with engagement overlays.
- **Stories row** — Instagram-style horizontal municipal account highlights.
- **Suggested follows** — dismissible municipal page recommendations.
- **Filter drawer** — animated 75% side-drawer for filtering/sorting.

Filter options:

- all, trending, following, high_priority, resolved, my_posts, nearby.

Engagement actions:

- Upvote / downvote (double-tap-to-like with heart animation).
- Comment navigation.
- Share.
- Open issue detail.
- Municipal profile navigation.

7.3 Municipal Content

- Municipal pages are searchable, suggested, and followable.
- Page profile with Updates / About tabs.
- Official updates are created as issues authored by `MunicipalPage` type.
- Official update types: `Announcement`, `ProjectUpdate`, `Emergency`, `WorkCompletion`, `PublicNotice`.
- Municipal feed supports seen/unseen prioritization via `municipal_post_seen` table.
- Suggested follows widget fetches from `/api/municipal/suggested`.

7.4 Map Intelligence

Screen: `MapScreen`

- Live location acquisition (best effort, cached fallback).
- Optional nearby seed generation for demo data.
- Category + status filters.
- 50 km proximity filtering (when user location available).
- Marker severity styling + emergency marker hint.
- Heatmap-like circles by severity.
- Google Maps integration via `react-native-maps` (API key in app.json).

7.5 Issue Detail and Public Traceability

Screen: `IssueDetailScreen`

- Shows media, severity (NeonBadge), tags, priority score (PriorityMeter), metadata.
- Full status timeline.
- Resolution proof (before/after image + worker remarks) when resolved.
- Social actions (upvote/downvote/comment/share).
- Follow issue for update notifications.

7.6 Notifications

Frontend: polling-based + swipe interactions.
Backend: CRUD endpoints + read state + unread count.

Capabilities:

- Fetch all notifications with unread count.
- Mark one / all as read.
- Delete one / all.
- Auto-refresh intervals.
- Swipe-to-delete UX.
- Push notification listeners (foreground receive + tap response) registered in `App.tsx`.

7.7 Push Notifications (FCM)

- Device push tokens registered via `Notifications.getDevicePushTokenAsync()` (native FCM, not Expo push tokens).
- Tokens stored in `push_tokens` table with device type.
- Backend `NotificationService` delivers via Firebase Admin `sendEachForMulticast` (v13+ API).
- Invalid tokens cleaned up on known failure codes.
- Expo tokens are explicitly filtered out server-side.
- Android notification channel: max importance, vibration pattern `[0,250,250,250]`.

7.8 Promotional Engagement Scheduler

- `promoScheduler.js` — 12 engagement message templates promoting reporting, upvoting, leaderboard, daily challenges, weekend challenges.
- Sends to random 20% of active users, 2–3 times per day during safe hours (9 AM–8 PM).
- Auto-reschedules daily at midnight.
- Can be manually triggered via `sendDailyPromoToRandomUsers()`.
- Started automatically on server boot.

7.9 Gamification

- **Points:** earned for reporting (+10), issue resolution, etc.
- **10-Level progression:**

Level	Name	XP Required
1	New Reporter	0
2	Active Citizen	500
3	Civic Watcher	1,200
4	Problem Solver	2,500
5	Community Guardian	5,000
6	Street Legend	10,000
7	City Steward	20,000
8	Urban Architect	35,000

Level	Name	XP Required
9	Governor Elect	50,000
10	Civic Hero	75,000

- **Badges:** stored in `badges` table, awarded via notifications.
- **Leaderboard:** citizen ranking by points (`GET /api/gamification/leaderboard`).
- **Platform stats:** aggregate issue statistics (`GET /api/gamification/stats`).
- **Profile level:** progress bar representation on ProfileScreen.

7.10 Admin Operations

Screen: `AdminDashboard`

- Pull high-priority issue queue.
- Update status (`Acknowledged`, `InProgress`, `Resolved`, `Rejected`).
- Assign department and optional worker/deadline.
- View aggregate platform stats.

7.11 Field Worker Operations

Screen: `FieldWorkerDashboard`

- Load assigned tasks (`GET /api/workflows/assigned/:workerId`).
- Start work → `InProgress`.
- Complete work → `Resolved` with proof photo upload.
- Open native map directions to issue coordinates.

7.12 Profile and Settings

- **ProfileScreen:** user stats, badge browser, level progress, logout.
- **EditProfileScreen:** profile editing form.
- **SettingsScreen:** dark mode toggle (persisted to AsyncStorage, app is currently dark-only), support/legal links, danger zone actions.

---

8) API Surface (Complete — As Built)

8.1 Auth (`/api/auth`)

Method	Endpoint	Auth	Description
POST	`/api/auth/register`	None	Create new user account
POST	`/api/auth/login`	None	Email/password login, returns JWT
POST	`/api/auth/supabase-login`	None	Exchange Supabase session for app JWT

8.2 Issues (`/api/issues`)

Method	Endpoint	Auth	Description

Method	Endpoint	Auth	Description
GET	`/api/issues`	JWT	Community feed with filters/sorting
GET	`/api/issues/municipal-feed`	JWT	Municipal page feed with seen/unseen
POST	`/api/issues/:id/seen`	JWT	Mark municipal post as seen
POST	`/api/issues/seed-nearby`	JWT	Generate demo seed data near location
GET	`/api/issues/geojson`	JWT	GeoJSON export for map rendering
GET	`/api/issues/:id`	JWT	Single issue detail
POST	`/api/issues`	JWT	Create new issue report
PUT	`/api/issues/:id/upvote`	JWT	Toggle upvote
PUT	`/api/issues/:id/downvote`	JWT	Toggle downvote
PUT	`/api/issues/:id/follow`	JWT	Toggle issue follow
POST	`/api/issues/:id/comments`	JWT	Add comment
GET	`/api/issues/:id/comments`	JWT	Get comments for issue

8.3 Workflows (`/api/workflows`)

Method	Endpoint	Auth	Description
PUT	`/api/workflows/:id/status`	Admin	Update issue status
PUT	`/api/workflows/:id/assign`	Admin	Assign department/worker/deadline
PUT	`/api/workflows/:id/worker-update`	Worker/Admin	Worker status update with proof
GET	`/api/workflows/assigned/:workerId`	Worker/Admin	Get tasks assigned to worker

8.4 Gamification (`/api/gamification`)

Method	Endpoint	Auth	Description
GET	`/api/gamification/leaderboard`	JWT	Citizen ranking by points
GET	`/api/gamification/badges`	JWT	All available badges
GET	`/api/gamification/stats`	JWT	Platform issue statistics

8.5 Notifications (`/api/notifications`)

Method	Endpoint	Auth	Description
GET	`/api/notifications/unread-count`	JWT	Get unread notification count
GET	`/api/notifications`	JWT	Get all notifications

Method	Endpoint	Auth	Description
PUT	`/api/notifications/read-all`	JWT	Mark all as read
PUT	`/api/notifications/:id/read`	JWT	Mark single as read
DELETE	`/api/notifications/:id`	JWT	Delete single notification
DELETE	`/api/notifications`	JWT	Delete all notifications

8.6 Municipal Pages (`/api/municipal`)

Method	Endpoint	Auth	Description
POST	`/api/municipal/create`	Admin	Create municipal page
GET	`/api/municipal/search`	JWT	Search pages by query
GET	`/api/municipal/suggested`	JWT	Get suggested pages for user
GET	`/api/municipal/:id`	JWT	Get page details
GET	`/api/municipal/:id/followers`	JWT	Get page followers
POST	`/api/municipal/:id/follow`	JWT	Follow page
POST	`/api/municipal/:id/unfollow`	JWT	Unfollow page
POST	`/api/municipal/:id/post`	Admin	Create official post on page
PATCH	`/api/municipal/:id`	Admin	Update page details

8.7 Users (`/api/users`)

Method	Endpoint	Auth	Description
GET	`/api/users/check-username/:username`	JWT	Check username availability
GET	`/api/users/profile`	JWT	Get current user profile
PUT	`/api/users/profile`	JWT	Update profile
POST	`/api/users/push-token`	JWT	Register device push token

Total: 33 endpoints across 7 route modules.

---

9) Data Model (Supabase PostgreSQL)

9.1 Primary Tables

Table	Key Columns	Purpose

Table	Key Columns	Purpose
`users`	id (UUID), name, email, password_hash, role, points, badges[], reports_count, reports_resolved, impact_score, region, avatar, department, account_status, username, city, ward, interests[], last_login_at	User accounts and profile
`levels`	level (PK int), name, xp_required	10-level gamification reference
`badges`	id (text PK), name, icon, description, points_required	Badge definitions
`issues`	id (UUID), user_id, municipal_page_id, author_type, official_update_type, title, description, image, video, location_address, location_coords (GEOGRAPHY), location_longitude, location_latitude, department_tag, status, category, priority_score, ai_severity, ai_tags[], anonymous, emergency, assigned_to, resolved_by, deadline	Civic issues and municipal posts
`issue_upvotes`	issue_id + user_id (composite PK)	Upvote records
`issue_downvotes`	issue_id + user_id (composite PK)	Downvote records
`issue_followers`	issue_id + user_id (composite PK)	Issue follow subscriptions
`status_timeline`	id (UUID), issue_id, status, comment, updated_by, dept	Status change audit trail
`resolution_proofs`	id (UUID), issue_id (unique), after_image, worker_remarks, resolved_at, resolved_by	Resolution evidence
`comments`	id (UUID), issue_id, user_id, text, likes, parent_comment_id	Comments with nesting
`notifications`	id (UUID), user_id, type, title, description, read, action_url	In-app notification records
`follows`	id (UUID), follower_id → users, following_id → municipal_pages, notifications_enabled	User → Municipal Page follows
`rewards`	id (UUID), user_id, points, reason	Point transaction log

Table	Key Columns	Purpose
`push_tokens`	id (UUID), user_id, token (unique), device_type (android/ios/web)	FCM push tokens
`municipal_pages`	id (UUID), name, handle (unique), department, region (JSONB), verified, followers_count, avatar, cover_image, description, created_by_admin_id, contact_email, is_active, page_type	Municipal authority profiles
`municipal_post_seen`	id (UUID), user_id + issue_id (unique), seen_at	Feed seen tracking

9.2 Geospatial Model

- `issues.location_coords` → `GEOGRAPHY(Point, 4326)` with `GIST` index.
- Also stores `location_longitude` and `location_latitude` as `DOUBLE PRECISION` for direct usage.

9.3 Indexes (Core + Performance)

- **Core:** `idx_users_email`, `idx_users_role`, `idx_issues_status`, `idx_issues_severity`, `idx_issues_user`, `idx_issues_status_severity`, `idx_issues_created`, `idx_issues_location` (GIST), `idx_timeline_issue`, `idx_comments_issue`, `idx_notifications_user`, `idx_notifications_user_read`.
- **Performance (migration):** `idx_issues_municipal_page_created`, `idx_issues_user_created`, `idx_issues_assigned_to`, `idx_issues_status_created`, `idx_issue_upvotes_issue`, `idx_issue_downvotes_issue`, `idx_issue_followers_issue`, `idx_issue_followers_user`, `idx_follows_follower`, `idx_follows_following`, `idx_comments_issue_created`, `idx_status_timeline_issue_created`.
- **Username:** `idx_users_username` (unique, where not null), `idx_users_username_lower`.
- **Push tokens:** `idx_push_tokens_user`, `idx_push_tokens_token`.
- **Municipal seen:** `idx_municipal_post_seen_user`, `idx_municipal_post_seen_issue`, `idx_municipal_post_seen_seen_at`.

9.4 Triggers

- `update_updated_at_column()` — PL/pgSQL function updating `updated_at` on row modification.
- Applied to: `users`, `issues`, `municipal_pages`, `push_tokens`.

9.5 Referential Integrity

- Foreign keys with `ON DELETE CASCADE` on engagement tables (upvotes, downvotes, followers, comments, notifications, follows, rewards, push_tokens, status_timeline, resolution_proofs, municipal_post_seen).
- Composite unique constraints on vote/follow tables to prevent duplicates.

---

10) Data Access Layer — `store.js` (48 Exports)

Token / Utility

`generateToken(userId)`, `JWT_SECRET`, `getLevelInfo(points)`

Users

`getUserById`, `getUserByEmail`, `getUserByUsername`, `isUsernameAvailable`, `createUser`, `updateUser`, `getCitizensLeaderboard`

Issues

`getIssues`, `getIssueById`, `createIssue`, `updateIssue`, `getIssueUpvotes`, `toggleUpvote`, `getIssueDownvotes`, `toggleDownvote`, `getIssueFollowers`, `toggleIssueFollow`, `addStatusTimeline`, `getStatusTimeline`, `setResolutionProof`, `getResolutionProof`, `getIssuesGeoJSON`

Comments

`getCommentsByIssue`, `getCommentCountByIssue`, `createComment`

Notifications

`getNotifications`, `createNotification`, `markNotificationsRead`, `markNotificationRead`, `deleteNotification`, `deleteAllNotifications`, `getUnreadNotificationCount`, `addPushToken`, `getPushTokens`, `deletePushToken`, `getAllPushTokensExcept`, `getAllActiveUserIds`

Badges

`getAllBadges`

Municipal Pages

`getMunicipalPageById`, `getMunicipalPageByHandle`, `createMunicipalPage`, `updateMunicipalPage`, `searchMunicipalPages`, `getSuggestedPages`

Follows

`isFollowing`, `addFollow`, `removeFollow`, `getFollowerIds`, `getFollowingCount`, `getFollowingPageIds`, `getSeenMunicipalPostIds`, `markMunicipalPostSeen`

Stats

`getIssueStats`, `getUserIssueCount`, `getUserResolvedCount`

Raw

`supabase` — direct Supabase client export.

Retry Mechanism: `withRetry()` wrapper — 2 retries with exponential backoff on transient errors.

---

11) Frontend API Client — `api.ts` (31 Wrapper Functions)

Namespace	Functions

Namespace	Functions
`authAPI`	`login`, `register`, `supabaseLogin`
`issuesAPI`	`getFeed`, `getMunicipalFeed`, `markMunicipalSeen`, `getById`, `create`, `upvote`, `downvote`, `followIssue`, `getComments`, `addComment`, `seedNearby`
`workflowAPI`	`updateStatus`, `assign`, `workerUpdate`, `getAssigned`
`notificationsAPI`	`getAll`, `getUnreadCount`, `markAllRead`, `markRead`, `deleteOne`, `deleteAll`
`userAPI`	`getProfile`, `updateProfile`, `checkUsername`, `registerPushToken`
`gamificationAPI`	`getLeaderboard`, `getBadges`, `getStats`

Default export: raw `api` Axios instance.
Type export: `User` interface (`_id`, `name`, `email`, `role`).

---

12) Role-Based Use Cases and Expected Outcomes

12.1 Citizen — Core Use Cases

Use Case A: Submit a civic issue

- Flow: Report → title/category/location/media → submit.
- Outcome: issue status `Submitted`, timeline initialized, +10 points (non-anonymous), appears in feed/map, notification may be generated.

Use Case B: Submit emergency issue

- Outcome: `emergency=true`, severity skewed toward critical, visually emphasized in map/feed/detail.

Use Case C: Track progress

- Outcome: status timeline updates, notifications on changes/comments/upvotes, resolution proof visible when resolved.

Use Case D: Engage socially

- Outcome: upvote/downvote affect counters, comments visible in issue detail, follow issue for update tracking, double-tap-to-like in feed.

Use Case E: Explore municipal content

- Outcome: switch to municipal feed, see stories row, follow suggested pages, view official updates in page profile.

12.2 Admin / Super Admin — Core Use Cases

Use Case A: Triage queue

- Flow: open dashboard high-priority feed.
- Outcome: critical/high issues visible with priority scores.

Use Case B: Update status

- Flow: set Acknowledged / InProgress / Resolved / Rejected.
- Outcome: status changes, timeline appended, owner notified, points awarded on resolution.

Use Case C: Assign work

- Flow: assign department + optional worker + deadline.
- Outcome: assignment stored, auto-acknowledged if previously submitted, worker notified.

Use Case D: Municipal communication

- Flow: create page → publish official updates (Announcement/ProjectUpdate/Emergency/WorkCompletion/PublicNotice).
- Outcome: updates in municipal feed/profile, citizens can follow and consume.

12.3 Field Worker — Core Use Cases

Use Case A: Start assigned task

- Outcome: status → `InProgress`, timeline updated.

Use Case B: Complete with proof

- Flow: upload completion photo → resolve.
- Outcome: issue → `Resolved`, resolution proof stored, counters updated, citizen notified + rewarded.

Use Case C: Navigate to site

- Outcome: opens native map intent with issue coordinates.

12.4 Municipal Page Entity

- Represents official civic authority voice.
- Types: `Department`, `City`, `EmergencyAuthority`.
- Publishes official updates, gains followers, content appears in municipal feed and page profile.

---

13) Notification and Messaging Behavior

13.1 In-App Notifications

Generated for:

- Onboarding welcome.
- Badge award events.
- Report submission confirmations.
- Status updates (acknowledged/resolved/rejected).
- Upvotes / comments on user's issues.
- Worker assignments.
- Promotional engagement messages (via promo scheduler).

13.2 Push Notifications (FCM)

- **Registration:** `expo-notifications getDevicePushTokenAsync()` → native FCM token → `/api/users/push-token`.
- **Storage:** `push_tokens` table, unique per token, supports android/ios/web.
- **Delivery:** `NotificationService` → `firebase-admin sendEachForMulticast` (v13+ API).
- **Token hygiene:** Expo push tokens filtered out. Invalid FCM tokens cleaned on delivery failure.
- **Android channel:** max importance, vibration `[0,250,250,250]`.
- **Debug tools:** `check_tokens.js`, `debug_notifications.js`, `test_push.js`.

13.3 Promotional Notifications

- 12 engagement templates (points, challenges, leaderboard, quick tips, etc.).
- 2–3 promos/day during 9 AM – 8 PM to random 20% of active users.
- Scheduler auto-starts on server boot, auto-reschedules at midnight.

---

14) Security and Access Control

14.1 Backend Route Protection

- `protect` middleware: validates JWT, injects normalized user object (`req.user` with `id`, `name`, `role`).
- `admin` guard: allows `admin` + `super_admin`.
- `fieldWorker` guard: allows `field_worker` + `admin`.
- Password storage: `bcryptjs` hashing.
- Request logging: masks passwords in body logs.

14.2 Configuration Security

- Firebase credentials: env-var `FIREBASE_SERVICE_ACCOUNT` (production) or local `serviceAccountKey.json` (dev).
- Supabase service role key: env-var only, validated on boot.
- JWT secret: from `JWT_SECRET` env var, with dev fallback `dev_secret_key_123`.
- Frontend Supabase uses publishable/anon key (appropriate for client apps with RLS).

14.3 Security Considerations

- SQL schema has proper FK cascade constraints and unique constraints.
- Supabase RLS policies should be validated for production-grade data isolation.
- `multer` restricts uploads to image/video MIME types, 50 MB limit.
- CORS is enabled globally (wide-open in current config).
- API responses use aggressive no-cache headers on `/api/` paths.

---

15) AI Layer Status (Current vs Intended)

15.1 Current Implementation (Mock)

All 3 AI modules in `backend/services/ai/` return static/random results:

Module	Behavior
`detectIssue.js`	Returns random category (Pothole/Garbage/StreetLight/Graffiti) with 95% confidence. Always `isValid: true`.
`severityScore.js`	Static category→severity map (Pothole→Medium, Garbage→Low, StreetLight→High, Graffiti→Low, default→Medium).
`duplicateCheck.js`	Always returns `{ isDuplicate: false }`.

15.2 Planned AI Architecture (from AIML_MODELS_PLAN.md)

The AIML plan (v2.0, 1321 lines) defines 12 models across 6 priority tiers:

Priority	Model	Architecture	Purpose
P0	Object Detection	YOLOv8 (640×640, 10 classes)	Detect civic issue types in images
P0	Severity Assessment	Dual-input CNN+MLP hybrid	Score 0–100 + level
P1	Image Validation	MobileNetV3/EfficientNet-B0	Reject blur/screenshot/indoor photos
P1	Duplicate Detection	Multi-modal (geo + CLIP + Sentence-BERT)	Cluster similar reports
P2	Smart Priority Scoring	XGBoost/NN regressor	8-factor priority formula
P2	Department Routing	Multi-class classifier + rules	Auto-route to department
P3	Resolution Verification	Siamese network	Before/after comparison
P3	NLP Text Analysis	BERT/MuRIL	Multi-language (EN/Hindi + Indian langs)
P4	Hotspot Detection	DBSCAN spatial clustering	Identify problem clusters
P4	Fraud/Fake Detection	Anomaly detection ensemble	Flag suspicious reports
P5	ETA Prediction	XGBoost/LightGBM	Predict resolution time
P5	Resource Optimization	VRP / Reinforcement Learning	Optimize worker routing

Target architecture: Mobile App → Node.js backend → Python FastAPI AI Service → ONNX/TensorRT inference.
Databases: PostgreSQL (Supabase) + Redis (cache) + pgvector/Milvus (embeddings).
Estimated cost: $750–1,400/month cloud infrastructure.
Roadmap: 4 phases, 32 weeks (Foundation → Enhancement → Intelligence → Optimization).

---

16) Observed Operational Characteristics

1. **Graceful fallback patterns** — dummy municipal posts, cached location, polling re-sync on API failure.
2. **Role-driven navigation** — strongly implemented with 3 distinct tab experiences + gated setup flow.
3. **Feature completeness is high for MVP+** — full citizen/admin/worker lifecycle, social engagement, municipal pages, gamification, push notifications, map intelligence.
4. **Premium UI polish** — reanimated animations, glassmorphism, neon badges, gradient effects, double-tap-to-like, reels/stories UX patterns.
5. **Developer tooling** — request logger with color-coded output, debug notification scripts, seed scripts for demo data, check_tokens diagnostic.
6. **Error resilience** — ErrorBoundary wrapping, global non-fatal error handler, suppressed LogBox warnings, store retry mechanism with exponential backoff.
7. **Promotional engagement** — automated promo scheduler for user retention.
8. **Some hardcoded/dev-oriented values** — LAN IP, demo municipal content, static badge definitions fallback, dev JWT secret fallback.

---

17) Expected End-to-End Scenario Walkthroughs

17.1 Citizen Report to Closure

1. Citizen opens app → onboarding → login (email/OTP/Google).
2. Completes location setup + profile setup (username, city, ward, interests).
3. Taps Report FAB → selects category, adds photo + description.
4. Location auto-detected (EXIF → GPS → cache).
5. Issue created (`Submitted`), +10 points, timeline initialized.
6. Issue appears in community feed + map with severity marker.
7. Admin sees issue in dashboard priority queue → acknowledges → assigns roads team + worker + deadline.
8. Worker receives assignment notification → starts work (`InProgress`) → navigates to site.
9. Worker completes → uploads proof photo → resolves (`Resolved`).
10. Citizen sees timeline progression, before/after evidence, resolution notification.
11. Points awarded to citizen and worker. Badges may unlock.

17.2 Municipal Communication Cycle

1. Admin creates municipal page (name, handle, department, region, page type).
2. Admin publishes official update post (Announcement/ProjectUpdate/etc.).
3. Post appears in municipal feed with official update type badge.
4. Citizens follow the page → updates in their municipal feed.
5. Stories row shows municipal page highlights.
6. Citizens can track seen/unseen posts.

17.3 Worker Task Execution Cycle

1. Worker opens assigned tasks dashboard → sees list from `/workflows/assigned/:workerId`.
2. Starts work → status `InProgress`, timeline updated.

3. Navigates to site via map intent.
4. Completes → uploads proof photo, adds remarks → status `Resolved`.
5. Resolution proof stored, worker counters updated, citizen receives notification + points.

17.4 Social Engagement Cycle

1. Citizen browses community feed → double-taps post to upvote (heart animation).
2. Scrolls reels tab for immersive issue browsing.
3. Opens issue detail → adds comment → follows issue.
4. Issue owner receives upvote + comment notifications.
5. Citizen checks leaderboard → sees rank and badge progress.

---

18) Non-Functional Expectations (Current State)

- **Responsiveness:** Polling + optimistic UI patterns. Spring/timing animations via Reanimated.
- **Reliability:** Retry/fallback for location (3 attempts), store retry with exponential backoff, error boundary catching.
- **Scalability readiness:** Modular route/data layers, performance indexes, paginated queries.
- **Auditability:** Status timeline provides transparent change history per issue.
- **Observability:** Request logger with method/path/body/user/status/duration. Dev logger with categories/timestamps.
- **Offline resilience:** AsyncStorage caching for user, token, location, onboarding state.

---

19) Known Gaps / Improvement Opportunities

1. **AI engine is mocked** — all 3 services return static/random results; replace with Python FastAPI inference per AIML plan.
2. **Legacy Mongoose models** — 6 model files exist in `backend/models/` but are unused (Supabase is the active DB). Should be removed or clearly archived.
3. **Legacy `config/db.js`** — Mongoose/MongoDB connector unused (app uses Supabase). Dead code.
4. **CORS wide-open** — `cors()` with no origin restrictions; should be tightened for production.
5. **JWT secret fallback** — `dev_secret_key_123` hardcoded in `store.js`; must be removed for production.
6. **Dark mode only** — settings toggle exists but light mode is not implemented.
7. **Frontend demo data / hardcoded values** — LAN IP, demo municipal content, static badge definitions should be externalized.
8. **Supabase RLS** — Row-Level Security policies should be validated and hardened.
9. **Empty `backend/eas.json`** — placeholder with no build profiles.
10. **No automated tests** — test script is a no-op (`echo "Error: no test specified"`).
11. **No CI/CD pipeline** documented.
12. **Notification service** has iterative comments/cleanup opportunities.
13. **`department_admin` role** is defined in schema but has no distinct UX or route guard separate from `admin`.

---

20) Module-by-Module Inventory Checklist

Backend Modules

Module	Status	File Count
Auth routes	☑	1
Issue routes	☑	1
Workflow routes	☑	1
Gamification routes	☑	1
Notification routes	☑	1
Municipal routes	☑	1
User routes	☑	1
Auth middleware	☑	1
Request Logger middleware	☑	1
Store data access layer	☑	1 (772 lines, 48 exports)
Seed data script	☑	1
Notification service + Firebase config	☑	2
Promo scheduler service	☑	1
AI mock services	☑ (mock)	3
Multer upload config	☑	1
Supabase client config	☑	1
SQL migrations	☑	5
Seed scripts	☑	3 (seed.js, seed_community, seed_municipal)
Debug/test utilities	☑	3 (check_tokens, debug_notifications, test_push)
Migration runners	☑	2 (run_migration, run_push_migration)
Legacy Mongoose models	⚠ unused	6
Legacy MongoDB config	⚠ unused	1

Frontend Modules

Module	Status	File Count
App entry (App.tsx + index.js)	☑	2
Auth flow screens	☑	8 screens
Main screens	☑	9 screens
Admin screens	☑	1

Module	Status	File Count
Worker screens	☑	1
Municipal screens	☑	1
Role-based navigators	☑	4 files
AuthContext	☑	1
Feed components	☑	5 components
Municipal components	☑	1
UI components	☑	6 components
API service wrappers	☑	1 (31 functions)
Location service	☑	1
Notification service	☑	1
Supabase client	☑	1
Theme/design tokens	☑	1
Logger utility	☑	1
Type declarations	☑	1
Config files	☑	5 (app.json, eas.json, tsconfig, package, google-services)

Documentation

Document	Status
Full App System Report	☑ (this document)
AI/ML Models Plan (v2.0)	☑ (1321 lines, 12 models)

---

21) Complete File Inventory

Root (4 files)

```
app.json
eas.json
package.json
.gitignore
```

Backend (33 files + node_modules)

```
server.js
package.json
```

```
package-lock.json
.env
.gitignore
eas.json
config/db.js
config/firebase.js
config/multer.js
config/serviceAccountKey.json
config/supabase.js
data/seed.js
data/store.js
middleware/authMiddleware.js
middleware/requestLogger.js
models/Comment.js
models/Follow.js
models/Gamification.js
models/Issue.js
models/MunicipalPage.js
models/User.js
routes/authRoutes.js
routes/gamificationRoutes.js
routes/issueRoutes.js
routes/municipal.js
routes/notificationRoutes.js
routes/userRoutes.js
routes/workflowRoutes.js
services/ai/detectIssue.js
services/ai/duplicateCheck.js
services/ai/severityScore.js
services/notificationService.js
services/promoScheduler.js
supabase_migration.sql
migration_push_tokens.sql
migration_performance_indexes.sql
migration_municipal_feed_seen.sql
add_username_columns.sql
run_migration.js
run_push_migration.js
seed_community_posts.js
seed_municipal.js
check_tokens.js
debug_notifications.js
test_push.js
public/images/garbage.avif
public/uploads/mcd-garbage_*.avif
```

Frontend (44+ source files)

```
App.tsx
index.js
app.json
```

```
eas.json
package.json
tsconfig.json
google-services.json
src/context/AuthContext.tsx
src/navigation/AdminTabs.tsx
src/navigation/CitizenTabs.tsx
src/navigation/RootNavigator.tsx
src/navigation/WorkerTabs.tsx
src/screens/Auth/OnboardingScreen.tsx
src/screens/Auth/LoginScreen.tsx
src/screens/Auth/RegisterScreen.tsx
src/screens/Auth/MagicLinkScreen.tsx
src/screens/Auth/OTPVerifyScreen.tsx
src/screens/Auth/LocationSetupScreen.tsx
src/screens/Auth/ProfileSetupScreen.tsx
src/screens/Auth/SplashScreen.tsx
src/screens/Main/HomeFeed.tsx
src/screens/Main/MapScreen.tsx
src/screens/Main/ReportIssueScreen.tsx
src/screens/Main/IssueDetailScreen.tsx
src/screens/Main/NotificationsScreen.tsx
src/screens/Main/ProfileScreen.tsx
src/screens/Main/LeaderboardScreen.tsx
src/screens/Main/SettingsScreen.tsx
src/screens/Main/EditProfileScreen.tsx
src/screens/Admin/AdminDashboard.tsx
src/screens/Worker/FieldWorkerDashboard.tsx
src/screens/Municipal/MunicipalProfileScreen.tsx
src/components/feed/FeedPost.tsx
src/components/feed/FeedToggle.tsx
src/components/feed/FilterDrawer.tsx
src/components/feed/ReelsTab.tsx
src/components/feed/StoriesRow.tsx
src/components/Municipal/SuggestedFollows.tsx
src/components/ui/ErrorBoundary.tsx
src/components/ui/FloatingReportFAB.tsx
src/components/ui/GlassCard.tsx
src/components/ui/IssueCard.tsx
src/components/ui/NeonBadge.tsx
src/components/ui/PriorityMeter.tsx
src/services/api.ts
src/services/locationService.ts
src/services/notificationService.ts
src/services/supabaseClient.tsx
src/theme/colors.ts
src/types/expo-vector-icons.d.ts
src/utils/logger.ts
assets/mcd-garbage_*.avif
android/ (native build output)
```

Docs (3 files)

```
FULL_APP_SYSTEM_REPORT.md
FULL_APP_SYSTEM_REPORT.pdf
AIML_MODELS_PLAN.md
```

---

22) Environment Variables Reference

Backend (`.env`)

Variable	Purpose
`PORT`	Server port (default 5000)
`SUPABASE_URL`	Supabase project URL
`SUPABASE_SERVICE_ROLE_KEY`	Supabase service role key (full access)
`JWT_SECRET`	Secret for signing app JWTs
`FIREBASE_SERVICE_ACCOUNT`	JSON string of Firebase service account (production)
`MONGO_URI`	Legacy (unused) — MongoDB connection string

Frontend (hardcoded in source)

Value	Location	Purpose
Supabase URL	`supabaseClient.tsx`	Supabase project URL
Supabase Anon Key	`supabaseClient.tsx`	Public key for client auth
Google Maps API Key	`app.json`	Maps rendering
LAN IP	`api.ts`	Dev base URL
Production URL	`api.ts`	Render deployment URL

---

23) Final Assessment

UrbanFix AI is a **substantially implemented civic operations platform** with a clear role-segmented architecture and full issue lifecycle support from reporting to resolution proof.

Strengths:

- Complete role-based operations (citizen/admin/worker) with distinct navigation.
- Full issue workflow: report → triage → assign → resolve → proof.
- Rich social engagement: feed, reels, stories, double-tap-to-like, comments, follows.
- Map intelligence with PostGIS geospatial queries + severity visualization.
- 10-level gamification with points, badges, and leaderboard.
- Push notifications via native FCM with automated promotional scheduling.
- Municipal page system with official update types and seen/unseen tracking.

- Premium UI with animations, glassmorphism, neon effects.
- Robust data layer with 48 store functions, retry mechanism, and performance indexes.
- Comprehensive SQL schema with proper FK constraints, triggers, and indexes.

Key evolution areas:

1. Replace 3 mock AI services with production inference (per AIML plan — 12 models, 32-week roadmap).
2. Remove legacy Mongoose models and `config/db.js` dead code.
3. Harden security: remove JWT fallback secret, restrict CORS, validate Supabase RLS.
4. Implement `department_admin` role with distinct permissions if needed.
5. Add light mode implementation for the existing dark mode toggle.
6. Externalize hardcoded dev values (LAN IP, demo data).
7. Establish automated testing and CI/CD pipeline.
8. Audit and update `FULL_APP_SYSTEM_REPORT.pdf` to match this revision.

---

24) Appendix: Key Concepts Dictionary

Term	Definition
Issue	Any reported civic problem or official municipal update record.
Status Timeline	Ordered history of status transitions with comments and updater info.
Resolution Proof	Post-resolution media (after image) + worker remarks evidence.
Municipal Page	Admin-created official authority profile for public updates.
Following (Issue)	User subscribes to status update notifications for an issue.
Following (Page)	User subscribes to municipal page updates in feed.
Priority Score	Numeric urgency signal (0–10) used in queue ranking and visualization.
AI Severity	Classified severity bucket: Low / Medium / High / Critical.
Official Update Type	Category of municipal post: Announcement / ProjectUpdate / Emergency / WorkCompletion / PublicNotice.
Promo Notification	Automated engagement message sent to random users for retention.
FAB	Floating Action Button — central report button in citizen tab bar.
Reels	TikTok-style vertical swipeable full-screen issue card experience.
Stories	Instagram-style horizontal municipal account highlights.
GlassCard	Glassmorphism-styled card wrapper (blur + transparency).
NeonBadge	Severity indicator with neon glow border effect.

End of report.

Revision: 2026-03-04 | **Previous:** 2026-02-20